

Spécification de collecte — schéma de stockage agnostique (multi-OS)

Données **brutes** à collecter par serveur, schéma **unique** valable AIX · Linux (toutes distros) · Windows — tous de plein droit. Voir aussi `field-applicability-matrix.md` et `field-rationale-and-ansible.md`.

Cadrage

Cette spécification décrit **uniquement la collecte brute** (premier et second passages) de métadonnées de stockage, sans interprétation ni calcul dérivé. Tous les champs calculés (classe de stockage, pourcentages, taux de croissance, écarts, sévérité...) sont **exclus** de la collecte et produits en aval par le pipeline (voir §6).

Principes de cadrage :

- **Un schéma unique multi-OS (v2)** : un seul document JSON par hôte, structurellement identique quelle que soit la plateforme. La présence des champs est **conditionnelle / nullable** selon l'OS et le type de filesystem (matrice d'applicabilité), jamais via des schémas divergents.
- **AIX + Linux (toutes distributions) + Windows traités AU MÊME NIVEAU**. Windows est **first-class dès maintenant** : même profondeur, même niveau d'obligation, mêmes objectifs couverts que AIX et Linux. **Il n'y a pas de « phase 2 »** : la collecte Windows (WinRM + PowerShell) et les standards Windows sont livrés en même temps que les autres.
- **Périmètre fleet** : ≥ 500 serveurs, AIX + RHEL / Oracle Linux / Rocky / AlmaLinux / Debian / Ubuntu / SLES + Windows Server.
- **Métadonnées non sensibles uniquement** : capacités, topologie, options de montage, résumé ACL non sensible. Aucun contenu de fichier, aucun ACL nominatif détaillé.
- **La commande de collecte se branche par plateforme** (`platform`) mais alimente le même schéma cible.
- **Livraison : un seul fichier JSON par passage de collecte** — un objet enveloppe `{ snapshot_id, generated_at, host_count, hosts: [ ... ] }` où chaque élément de `hosts[]` est le document d'un serveur (le schéma ci-dessous). Pas un fichier par serveur. (JSONL — un objet-hôte par ligne — accepté pour les très grands parcs.)

Principes

1. **Octets partout**. Toutes les tailles (`size_*` , `reserved_bytes` , `extent_size_bytes` , paging...) sont exprimées en **octets**, jamais en blocs, Ko, Mo, Go ni en pourcentage. Les sorties natives en Ko (`df -k` , `lspv` , `vgs --units b`) ou en blocs sont normalisées en octets à la collecte.
2. **Modèle agnostique : `filesystems[]` + `storage_topology`**.
 - `filesystems[]` est le **dénominateur universel** : il couvre à la fois les points de montage Unix (`/` , `/var` ...) ET les volumes Windows (`C:` , `D:` , points de montage en dossier). C'est la seule

structure obligatoire sur tous les OS.

- `storage_topology` décrit la **topologie physique/logique** en niveaux **neutres** (`groups` / `volumes` / `disks`) assortis d'un discriminateur `*_kind` . Elle est **vide** pour un Linux en partitions simples ou un Windows en disque basique sans pool.
- 3. **Lien par clés `group_ref` / `backing_ref` , pas par imbrication.** Un filesystem référence sa volume/groupe sous-jacent par **clé** (`backing_ref` , `group_ref`), et non par nesting. Ce couplage faible permet au pipeline de **dériver `storage_class`** (système vs données) de façon **généralisée** : rootvg/datavg sous AIX, VG système/données sous Linux, `C:` (système) vs autres volumes (données) sous Windows.
- 4. **Collecte conditionnelle branchée par plateforme.** La commande lit `platform` puis exécute la branche adéquate (`lsvg` / `lslv` / `lspv` / `df` / `lspv` ; `vgs` / `lvs` / `pvs` / `df` / `tune2fs` / `btrfs` ; PowerShell `Get-*` / `Win32_PageFileUsage` / `Get-Acl`). Les champs non applicables sont mis à `null` , jamais silencieusement supprimés.
- 5. **Série temporelle UTC.** Chaque document est un **snapshot** horodaté (`scan_timestamp` en **UTC ISO-8601**, `snapshot_id`). Les documents successifs par hôte forment une série temporelle permettant la dérivation de croissance et de prévision en aval.
- 6. **Métadonnées non sensibles.** Seuls capacité, topologie, options de montage et **résumé ACL non sensible** (`acl_summary` Windows) sont collectés.
- 7. **Nuances par OS (déterminantes pour la justesse des octets) :**
 - **AIX** : LVM (rootvg/datavg), JFS2 ; `lsvg` / `lslv` / `lspv` , `df -k` / `df -v` , `lspv -a` . **Pas de blocs réservés ext** → `reserved_bytes = null` .
 - **RHEL / Oracle Linux / Rocky / AlmaLinux** : **XFS par défaut** → **pas de `reserved_bytes`** (`null`) ; utiliser `df -i` pour les inodes et `xfs_info` . LVM fréquent (`vgs` / `lvs` / `pvs --units b`). `/boot/efi` présent **uniquement en UEFI x86_64**.
 - **Debian / Ubuntu** : **EXT4 par défaut** → `reserved_bytes` via `tune2fs -l` (blocs réservés × taille de bloc). LVM optionnel.
 - **SUSE / SLES** : **BTRFS par défaut avec sous-volumes + snapshots** → **df trompeur** ; utiliser `btrfs filesystem usage` / `qgroups` pour les octets réels. LVM optionnel.
 - **Windows** : **NTFS/ReFS**, lettres de lecteur / points de montage en dossier ; collecte **WinRM + PowerShell**. **Pas d'inodes** (`inodes_* = null`). Les **ACL remplacent les options de montage** (`access_model = ntfs-acl` , `mount_options = null` , `acl_summary` renseigné). **Storage Spaces** : pool = `group` (`storage-pool`), virtual disk = `volume` (`virtual-disk`). **Disque basique** : partitions exposées comme `volumes` (`partition`), `groups` vide. Tailles déjà en octets.

Schéma JSON cible

```
{
  // — Identité & métadonnées de collecte (UNIVERSEL, tous OS) —————
  "host_id":           "string",    // OBL – identifiant stable de l'hôte
  "hostname":          "string",    // OBL
  "gsma_code":         "string",    // OBL – code site/parc
```

```

"host_role_env":    "string",    // OBL – prod | preprod | test | dev ...
"platform":        "string",    // OBL – enum: aix | linux | windows
"os_family":       "string",    // OBL – AIX | RedHat | Debian | Suse | OracleLinux | Windows
"os_distribution":  "string",    // OBL – AIX | RHEL | Ubuntu | Debian | SLES | OracleLinux |
WindowsServer
"os_version":      "string",    // OBL
"architecture":    "string",    // OBL – ppc64 | x86_64 | ...
"snapshot_id":     "string",    // OBL – identifiant du snapshot
"scan_timestamp":  "string",    // OBL – UTC ISO-8601
"collection_method": "string",  // OBL – ssh-shell | winrm-powershell | agent ...
"scan_confidence": "number",    // OBL – 0..1 fiabilité de la collecte

// — DÉNOMINATEUR UNIVERSEL : tout filesystem montable / volume Windows —
"filesystems": [{
  "mount_point":    "string",    // OBL – "/", "/var" | "C:", "D:" | mount-folder
  "label":          "string|null", // REC
  "fstype":         "string",    // OBL – xfs|ext4|btrfs|jfs2|ntfs|refs|vfat...
  "size_total_bytes": "number",    // OBL – octets
  "size_used_bytes": "number",    // OBL – octets
  "size_available_bytes": "number", // OBL – octets
  "reserved_bytes": "number|null", // COND – ext uniquement ; null sur
xfs/jfs2/ntfs/btrfs
  "inodes_total":    "number|null", // COND – null sur Windows NTFS/ReFS
  "inodes_used":     "number|null", // COND – null sur Windows NTFS/ReFS
  "access_model":    "string",    // OBL – "posix-mount-options" | "ntfs-acl"
  "mount_options":   "string|null", // COND – flags Unix ; null sur Windows
  "acl_summary":     "object|null", // COND – résumé ACL non sensible Windows ; null sur
Unix
  "backing_kind":    "string",    // OBL – lvm-lv | partition | virtual-disk | storage-
space | network | pseudo
  "backing_ref":     "string|null", // OBL* – clé vers volume/disque sous-jacent
  "group_ref":       "string|null", // OBL* – clé vers le groupe (VG/pool)
  "content_aggregates": { /* object */ // REC – agrégats non sensibles (2e passage)
}],

// — TOPOLOGIE STOCKAGE PLATEFORME – niveaux neutres + discriminateur *_kind
"storage_topology": {
  "groups": [{
    "group_name":    "string",    // OBL (si groupe présent)
    "group_kind":    "string",    // OBL – lvm-vg | storage-pool
    "size_total_bytes": "number",  // OBL – octets
    "size_free_bytes": "number",   // OBL – octets
    "member_disk_count": "number", // OBL
    "state":         "string",    // OBL – active | online | degraded ...
    "extent_size_bytes": "number|null", // COND – PE size (LVM) / extent ; null si N/A

```

```

    "capabilities": {
        "max_lvs": "number|null", // COND – nullable selon OS
        "max_pps": "number|null", // AIX
        "vg_type": "string|null", // AIX (big | scalable ...)
        "resiliency": "string|null" // Windows (Mirror | Parity | Simple)
    }
}],
"volumes": [{
    "volume_name": "string", // OBL (si volume présent)
    "volume_kind": "string", // OBL – lvm-lv | partition | virtual-disk
    "group_ref": "string|null", // OBL – clé vers groupe parent
    "size_bytes": "number", // OBL – octets
    "type": "string|null", // REC – jfs2 | linear | striped | mirror | gpt-part
    ...
    "redundancy": "string|null" // COND – none | mirror | parity ...
}],
"disks": [{
    "disk_name": "string", // OBL (si disque exposé)
    "size_bytes": "number", // OBL – octets
    "free_bytes": "number|null", // REC – octets
    "group_ref": "string|null", // REC – clé vers groupe
    "media_type": "string|null" // REC – hdd | ssd | unspecified
}]
},

// — PAGING / SWAP / PAGEFILE —————
"paging": [{
    "name": "string", // OBL
    "kind": "string", // OBL – aix-paging-lv | linux-swap-partition |
linux-swap-file | windows-pagefile
    "size_total_bytes": "number", // OBL – octets
    "size_used_bytes": "number|null" // REC – octets
}]
}

```

OBL* = obligatoire mais **nullable** quand la topologie n'existe pas (partition simple Linux, disque basique Windows) ; la clé est alors **null**.

Champs à collecter, par niveau

Identité & métadonnées de collecte

Champ	Type	Source AIX	Source Linux (LVM/df)	Source Windows (PowerShell)	Ob.
host_id	string	CMDB / <code>uname -f</code>	CMDB / <code>hostnamectl</code>	CMDB / <code>(Get-CimInstance Win32_ComputerSystemProduct).UID</code>	OBL
hostname	string	<code>hostname</code>	<code>hostname</code>	<code>\$env:COMPUTERNAME</code>	OBL
gsma_code	string	inventaire	inventaire	inventaire	OBL
host_role_env	string	inventaire	inventaire	inventaire	OBL
platform	enum	<code>aix</code>	<code>linux</code>	<code>windows</code>	OBL
os_family	enum	<code>uname -s</code> → AIX	<code>/etc/os-release</code> ID_LIKE	<code>Windows</code>	OBL
os_distribution	enum	AIX	<code>/etc/os-release</code> ID	<code>(Get-CimInstance Win32_OperatingSystem).Caption</code>	OBL
os_version	string	<code>oslevel -s</code>	<code>/etc/os-release</code> VERSION_ID	<code>[Environment]::OSVersion.Version</code>	OBL
architecture	string	<code>uname -p</code> / <code>prtconf</code>	<code>uname -m</code>	<code>\$env:PROCESSOR_ARCHITECTURE</code>	OBL
snapshot_id	string	généré collecteur	généré collecteur	généré collecteur	OBL
scan_timestamp	string (UTC ISO-8601)	<code>date -u</code>	<code>date -u</code>	<code>(Get-Date).ToUniversalTime().ToString('o')</code>	OBL
collection_method	string	<code>ssh-shell</code>	<code>ssh-shell</code>	<code>winrm-powershell</code>	OBL
scan_confidence	number 0..1	collecteur	collecteur	collecteur	OBL

filesystems[] (dénominateur universel)

Champ	Type	Source AIX	Source Linux (LVM/df)	Source Windows (PowerShell)	Ob.
mount_point	string	<code>df -k</code> / <code>lsfs</code>	<code>df -P</code> / <code>findmnt</code>	<code>Get-Volume .DriveLetter / Path</code>	OBL
label	string null	<code>lsfs</code>	<code>lsblk -o LABEL / blkid</code>	<code>Get-Volume .FileSystem Label</code>	REC
fstype	string	<code>lsfs</code> (jfs2)	<code>findmnt -o FSTYPE / df -T</code>	<code>Get-Volume .FileSystem (NTFS/ReFS)</code>	OBL

Champ	Type	Source AIX	Source Linux (LVM/df)	Source Windows (PowerShell)	Ob.
size_total_bytes	number	<code>df -k</code> ×1024	<code>df -PB1 / df --output=size</code> ×1024	<code>Get-Volume .Size</code>	OBL
size_used_bytes	number	<code>df -k</code> ×1024	<code>df -PB1 used</code>	<code>Size - SizeRemaining</code>	OBL
size_available_bytes	number	<code>df -k</code> ×1024	<code>df -PB1 avail</code>	<code>Get-Volume .SizeRemaining</code>	OBL
reserved_bytes	number null	null (JFS2)	ext: <code>tune2fs -l</code> (rsv blocks × block size) ; xfs/btrfs: null	null (NTFS/ReFS)	COND
inodes_total	number null	<code>df -v / istat</code>	<code>df -i</code>	null	COND
inodes_used	number null	<code>df -v</code>	<code>df -i</code>	null	COND
access_model	enum	<code>posix-mount-options</code>	<code>posix-mount-options</code>	<code>ntfs-acl</code>	OBL
mount_options	string null	<code>lsfs /etc/filesystems</code>	<code>findmnt -o OPTIONS</code>	null	COND
acl_summary	object null	null	null	<code>Get-Acl</code> → résumé non sensible (héritage, nb ACE, groupes well-known)	COND
backing_kind	enum	<code>lvm-lv</code> (jfs2)	<code>lvm-lv partition network pseudo</code>	<code>virtual-disk storage-space partition network</code>	OBL
backing_ref	string null	<code>lslv</code> nom LV	<code>lsblk</code> LV/part	<code>Get-Partition / Get-VirtualDisk id</code>	OBL*
group_ref	string null	<code>lslv</code> → VG	<code>lvs -o vg_name</code>	<code>Get-StoragePool</code> FriendlyName	OBL*
content_aggregates	object	2e passage (non sensible)	2e passage (non sensible)	2e passage (non sensible)	REC

BTRFS (SLES) : `size_*` issus de **btrfs filesystem usage** (et qgroups) et non de `df`, qui est trompeur avec sous-volumes/snapshots.

storage_topology.groups[] (lvm-vg | storage-pool)

Champ	Type	Source AIX	Source Linux (LVM/df)	Source Windows (PowerShell)	Ob.
group_name	string	<code>lsvg</code>	<code>vgs -o vg_name</code>	<code>Get-StoragePool .FriendlyName</code>	OBL
group_kind	enum	<code>lvm-vg</code>	<code>lvm-vg</code>	<code>storage-pool</code>	OBL

Champ	Type	Source AIX	Source Linux (LVM/df)	Source Windows (PowerShell)	Ob.
size_total_bytes	number	<code>lsvg</code> (TOTAL PPs × PP size)	<code>vgs -o vg_size -units b</code>	<code>Get-StoragePool .Size</code>	OBL
size_free_bytes	number	<code>lsvg</code> (FREE PPs × PP size)	<code>vgs -o vg_free -units b</code>	<code>Size -AllocatedSize</code>	OBL
member_disk_count	number	<code>lsvg -p</code>	<code>pvs -o vg_name vgs -o pv_count</code>	<code>Get-PhysicalDisk -StoragePool count</code>	OBL
state	string	<code>lsvg</code> (VG STATE)	<code>vgs -o vg_attr</code>	<code>Get-StoragePool .HealthStatus</code>	OBL
extent_size_bytes	number null	<code>lsvg</code> (PP SIZE)	<code>vgs -o vg_extent_size -units b</code>	null	COND
capabilities.max_lvs	number null	<code>lsvg</code> (MAX LVs)	null	null	COND
capabilities.max_pps	number null	<code>lsvg</code> (MAX PPs)	null	null	COND
capabilities.vg_type	string null	<code>lsvg</code> (big/scalable)	null	null	COND
capabilities.resiliency	string null	null	null	<code>Get-StoragePool .ResiliencySettingNameDefault</code>	COND

storage_topology.volumes[] (lvm-lv | partition | virtual-disk)

Champ	Type	Source AIX	Source Linux (LVM/df)	Source Windows (PowerShell)	Ob.
volume_name	string	<code>lslv</code> / <code>lsvg -l</code>	<code>lvs -o lv_name / lsblk</code> (part)	<code>Get-VirtualDisk .FriendlyName / Get-Partition</code>	OBL
volume_kind	enum	<code>lvm-lv</code>	<code>lvm-lv</code> <code>partition</code>	<code>virtual-disk</code> <code>partition</code>	OBL
group_ref	string null	<code>lslv</code> → VG	<code>lvs -o vg_name</code>	<code>Get-VirtualDisk</code> → pool ; null si partition basique	OBL
size_bytes	number	<code>lslv</code> (LPs × PP size)	<code>lvs -o lv_size -units b / lsblk -b SIZE</code>	<code>Get-VirtualDisk .Size / Get-Partition .Size</code>	OBL
type	string null	<code>lslv</code> (TYPE jfs2/...)	<code>lvs -o segtype</code> (linear/striped) / <code>gpt-part</code>	<code>Get-VirtualDisk .ProvisioningType</code> / part type	REC
redundancy	string null	<code>lslv</code> (COPIES → mirror)	<code>lvs -o lv_attr</code> (mirror/raid)	<code>Get-VirtualDisk .ResiliencySettingName</code>	COND

storage_topology.disks[]

Champ	Type	Source AIX	Source Linux (LVM/df)	Source Windows (PowerShell)	Ob.
disk_name	string	<code>lspv</code> (hdiskN)	<code>pvs -o pv_name / lsblk -d</code>	<code>Get-Disk .Number / Get-PhysicalDisk .FriendlyName</code>	OBL
size_bytes	number	<code>lspv</code> (TOTAL PPs × PP size)	<code>pvs -o pv_size -units b / lsblk -b -d SIZE</code>	<code>Get-Disk .Size / Get-PhysicalDisk .Size</code>	OBL
free_bytes	number null	<code>lspv</code> (FREE PPs × PP size)	<code>pvs -o pv_free -units b</code>	<code>Get-Disk non alloué</code>	REC
group_ref	string null	<code>lspv</code> → VG	<code>pvs -o vg_name</code>	<code>Get-PhysicalDisk - StoragePool .FriendlyName</code>	REC
media_type	string null	null	<code>lsblk -o ROTA</code> → hdd/ssd	<code>Get-PhysicalDisk .MediaType</code>	REC

paging[]

Champ	Type	Source AIX	Source Linux (LVM/df)	Source Windows (PowerShell)	Ob.
name	string	<code>lspcs -a</code> (Page Space)	<code>swapon -- show=NAME</code>	<code>Win32_PageFileUsage .Name</code>	OBL
kind	enum	<code>aix-paging-lv</code>	<code>linux-swap-partition linux-swap-file</code>	<code>windows-pagefile</code>	OBL
size_total_bytes	number	<code>lspcs -a</code> (MB ×1048576)	<code>swapon -- show=SIZE -- bytes</code>	<code>Win32_PageFileUsage .AllocatedBaseSize ×1MB</code>	OBL
size_used_bytes	number null	<code>lspcs -a</code> (%Used × size)	<code>swapon -- show=USED -- bytes</code>	<code>Win32_PageFileUsage .CurrentUsage ×1MB</code>	REC

content_aggregates (2e passage, non sensible) — filesystems[].content_aggregates

Champ	Type	Source AIX	Source Linux (LVM/df)	Source Windows (PowerShell)	Ob.
file_count	number null	<code>find -type f wc -l</code>	<code>find -type f wc -l</code>	<code>Get-ChildItem - Recurse - File .Count</code>	REC
dir_count	number null	<code>find -type d wc -l</code>	<code>find -type d wc -l</code>	<code>Get-ChildItem - Recurse - Directory .Count</code>	REC
largest_file_bytes	number null	<code>find -type f - printf</code>	<code>find -printf '%s' max</code>	<code>Measure-Object Length -Max</code>	REC

Champ	Type	Source AIX	Source Linux (LVM/df)	Source Windows (PowerShell)	Ob.
oldest_mtime / newest_mtime	string null (UTC)	<code>find -printf '%T@'</code>	<code>find -printf '%T@'</code>	<code>LastWriteTimeUtc min/max</code>	REC
ext_size_top (par extension)	object null	<code>find</code> + agrégat	<code>find</code> + agrégat	<code>Group-Object Extension / Sum Length</code>	REC

Les agrégats sont **strictement non nominatifs** (compteurs, tailles, dates ; jamais de noms ni de contenus de fichiers).

Couverture des objectifs

L'**ensemble obligatoire** (Identité + `filesystems[]` + `storage_topology` quand applicable + `paging[]`) suffit à couvrir les 5 objectifs sur **chaque OS**.

Objectif	Champs obligatoires porteurs	AIX	Linux	Windows
1. Détection d'écarts vs standards (FS + groupes)	<code>mount_point</code> , <code>fstype</code> , <code>size_total_bytes</code> , <code>group_*</code> , <code>volume_*</code>	OK	OK	OK
2. Optimisation (données inutiles/obsolètes/mal placées)	<code>size_used/available_bytes</code> , <code>group_ref / backing_ref</code> (→ system/data), <code>content_aggregates</code>	OK	OK	OK
3. Fiabilité & scalabilité (éviter FS pleins, croissance)	<code>size_total/used/available_bytes</code> , <code>inodes_*</code> (Unix), <code>scan_timestamp</code> (série)	OK	OK	OK
4. Efficacité opérationnelle (structures standard, prévision)	topologie neutre <code>groups/volumes/disk s</code> , <code>extent_size_bytes</code> , <code>paging[]</code> , série UTC	OK	OK	OK
5. Réduction du risque (données appropriées sur stockage système, conformité)	<code>group_ref / backing_ref</code> (→ <code>storage_class</code>), <code>access_model</code> , <code>acl_summary / mount_options</code>	OK	OK	OK

Inodes (objectif 3) sont `null` sous Windows : la saturation y est couverte par `size_*` (pas de notion d'inode). Le résumé ACL (objectif 5) remplace `mount_options` sous Windows. La couverture reste donc **complète et au même niveau** sur les trois plateformes.

Champs dérivés (exclus)

Les champs suivants **ne sont pas collectés** : ils sont **dérivés en aval** par le pipeline à partir des champs bruts. Listés ici pour cadrer la frontière collecte / traitement.

Champ dérivé	Dérivé de	Sémantique
storage_class	group_ref / backing_ref + règles OS	system vs data ; généralise rootvg/datavg (AIX), VG système/données (Linux) et C: = système / autres = données (Windows)
mount_category	mount_point + standards	rôle normalisé (/var , /tmp , D: data...)
used_percent	size_used_bytes / size_total_bytes	taux d'occupation
inode_used_percent	inodes_used / inodes_total	saturation d'inodes (Unix)
*_normalized	tailles brutes	valeurs normalisées (unités/standards)
growth_rate	série temporelle des snapshots	vitesse de croissance
projected_days_to_full	growth_rate + size_available_bytes	prévision de saturation
deviation_type	comparaison aux standards	type d'écart (FS / groupe / placement)
severity	deviation_type + seuils	criticité
confidence_score	scan_confidence + cohérence	confiance de l'analyse

Exemples JSON

AIX — rootvg (JFS2, LVM)

```
{
  "host_id": "AIX-PWR-0421-UUID",
  "hostname": "aixprd0421",
  "gsma_code": "FR-PAR-DC1",
  "host_role_env": "prod",
  "platform": "aix",
  "os_family": "AIX",
  "os_distribution": "AIX",
  "os_version": "7.2.5.4",
  "architecture": "ppc64",
  "snapshot_id": "snap-aix-20260612T020000Z-0421",
  "scan_timestamp": "2026-06-12T02:00:00Z",
  "collection_method": "ssh-shell",
  "scan_confidence": 0.99,
  "filesystems": [
    {
      "mount_point": "/",
      "label": null,
      "fstype": "jfs2",
      "size_total_bytes": 2147483648,
```

```

    "size_used_bytes": 1288490188,
    "size_available_bytes": 858993460,
    "reserved_bytes": null,
    "inodes_total": 524288,
    "inodes_used": 38211,
    "access_model": "posix-mount-options",
    "mount_options": "rw,log=/dev/hd8",
    "acl_summary": null,
    "backing_kind": "lvm-lv",
    "backing_ref": "hd4",
    "group_ref": "rootvg",
    "content_aggregates": {}
  },
  {
    "mount_point": "/data",
    "label": null,
    "fstype": "jfs2",
    "size_total_bytes": 107374182400,
    "size_used_bytes": 64424509440,
    "size_available_bytes": 42949672960,
    "reserved_bytes": null,
    "inodes_total": 13107200,
    "inodes_used": 902144,
    "access_model": "posix-mount-options",
    "mount_options": "rw,log=/dev/loglv00",
    "acl_summary": null,
    "backing_kind": "lvm-lv",
    "backing_ref": "datalv00",
    "group_ref": "datavg",
    "content_aggregates": {}
  }
],
"storage_topology": {
  "groups": [
    {
      "group_name": "rootvg",
      "group_kind": "lvm-vg",
      "size_total_bytes": 68719476736,
      "size_free_bytes": 21474836480,
      "member_disk_count": 1,
      "state": "active",
      "extent_size_bytes": 134217728,
      "capabilities": { "max_lvs": 256, "max_pps": 1016, "vg_type": "scalable", "resiliency":
null }
    },
  ],

```

```

{
  "group_name": "datavg",
  "group_kind": "lvm-vg",
  "size_total_bytes": 214748364800,
  "size_free_bytes": 107374182400,
  "member_disk_count": 2,
  "state": "active",
  "extent_size_bytes": 268435456,
  "capabilities": { "max_lvs": 512, "max_pps": 2032, "vg_type": "scalable", "resiliency":
null }
}
],
"volumes": [
  { "volume_name": "hd4", "volume_kind": "lvm-lv", "group_ref": "rootvg", "size_bytes":
2147483648, "type": "jfs2", "redundancy": "none" },
  { "volume_name": "datalv00", "volume_kind": "lvm-lv", "group_ref": "datavg", "size_bytes":
107374182400, "type": "jfs2", "redundancy": "mirror" }
],
"disks": [
  { "disk_name": "hdisk0", "size_bytes": 68719476736, "free_bytes": 21474836480, "group_ref":
"rootvg", "media_type": null },
  { "disk_name": "hdisk1", "size_bytes": 107374182400, "free_bytes": 53687091200, "group_ref":
"datavg", "media_type": null },
  { "disk_name": "hdisk2", "size_bytes": 107374182400, "free_bytes": 53687091200, "group_ref":
"datavg", "media_type": null }
]
},
"paging": [
  { "name": "hd6", "kind": "aix-paging-lv", "size_total_bytes": 8589934592, "size_used_bytes":
858993459 }
]
}

```

RHEL — XFS sans LVM (partitions simples, UEFI)

```

{
  "host_id": "RHEL-VM-1180-UUID",
  "hostname": "rhel9web1180",
  "gsma_code": "FR-LY0-DC2",
  "host_role_env": "prod",
  "platform": "linux",
  "os_family": "RedHat",
  "os_distribution": "RHEL",
  "os_version": "9.4",
  "architecture": "x86_64",

```

```
"snapshot_id": "snap-rhel-20260612T020500Z-1180",
"scan_timestamp": "2026-06-12T02:05:00Z",
"collection_method": "ssh-shell",
"scan_confidence": 0.98,
"filesystems": [
  {
    "mount_point": "/",
    "label": "root",
    "fstype": "xfs",
    "size_total_bytes": 53687091200,
    "size_used_bytes": 16106127360,
    "size_available_bytes": 37580963840,
    "reserved_bytes": null,
    "inodes_total": 26214400,
    "inodes_used": 184320,
    "access_model": "posix-mount-options",
    "mount_options": "rw,relatime,attr2,inode64,noquota",
    "acl_summary": null,
    "backing_kind": "partition",
    "backing_ref": "/dev/sda2",
    "group_ref": null,
    "content_aggregates": {}
  },
  {
    "mount_point": "/boot/efi",
    "label": null,
    "fstype": "vfat",
    "size_total_bytes": 629145600,
    "size_used_bytes": 12582912,
    "size_available_bytes": 616562688,
    "reserved_bytes": null,
    "inodes_total": null,
    "inodes_used": null,
    "access_model": "posix-mount-options",
    "mount_options": "rw,relatime,fmask=0077,dmask=0077",
    "acl_summary": null,
    "backing_kind": "partition",
    "backing_ref": "/dev/sda1",
    "group_ref": null,
    "content_aggregates": {}
  }
],
"storage_topology": {
  "groups": [],
  "volumes": [
```

```

    { "volume_name": "/dev/sda1", "volume_kind": "partition", "group_ref": null, "size_bytes":
629145600, "type": "gpt-part", "redundancy": null },
    { "volume_name": "/dev/sda2", "volume_kind": "partition", "group_ref": null, "size_bytes":
53687091200, "type": "gpt-part", "redundancy": null }
],
"disks": [
    { "disk_name": "/dev/sda", "size_bytes": 54760833024, "free_bytes": 0, "group_ref": null,
"media_type": "ssd" }
],
},
"paging": [
    { "name": "/dev/sda3", "kind": "linux-swap-partition", "size_total_bytes": 4294967296,
"size_used_bytes": 0 }
]
}

```

SLES — BTRFS avec sous-volumes (octets via `btrfs filesystem usage`)

```

{
  "host_id": "SLES-VM-3307-UUID",
  "hostname": "sles15app3307",
  "gsma_code": "FR-PAR-DC1",
  "host_role_env": "preprod",
  "platform": "linux",
  "os_family": "Suse",
  "os_distribution": "SLES",
  "os_version": "15-SP5",
  "architecture": "x86_64",
  "snapshot_id": "snap-sles-20260612T021000Z-3307",
  "scan_timestamp": "2026-06-12T02:10:00Z",
  "collection_method": "ssh-shell",
  "scan_confidence": 0.95,
  "filesystems": [
    {
      "mount_point": "/",
      "label": "ROOT",
      "fstype": "btrfs",
      "size_total_bytes": 42949672960,
      "size_used_bytes": 15032385536,
      "size_available_bytes": 27917287424,
      "reserved_bytes": null,
      "inodes_total": null,
      "inodes_used": null,
      "access_model": "posix-mount-options",
      "mount_options": "rw,relatime,space_cache=v2,subvolid=257,subvol=@/.snapshots/1/snapshot",

```

```

    "acl_summary": null,
    "backing_kind": "lvm-lv",
    "backing_ref": "lv_root",
    "group_ref": "system",
    "content_aggregates": {}
  }
],
"storage_topology": {
  "groups": [
    {
      "group_name": "system",
      "group_kind": "lvm-vg",
      "size_total_bytes": 53687091200,
      "size_free_bytes": 10737418240,
      "member_disk_count": 1,
      "state": "active",
      "extent_size_bytes": 4194304,
      "capabilities": { "max_lvs": null, "max_pps": null, "vg_type": null, "resiliency": null }
    }
  ],
  "volumes": [
    { "volume_name": "lv_root", "volume_kind": "lvm-lv", "group_ref": "system", "size_bytes":
42949672960, "type": "linear", "redundancy": "none" }
  ],
  "disks": [
    { "disk_name": "/dev/sda", "size_bytes": 53687091200, "free_bytes": 10737418240,
"group_ref": "system", "media_type": "ssd" }
  ]
},
"paging": [
  { "name": "/dev/system/lv_swap", "kind": "linux-swap-partition", "size_total_bytes":
2147483648, "size_used_bytes": 268435456 }
]
}

```

Windows — NTFS **c:** système + **d:** données + Storage Space + pagefile

```

{
  "host_id": "WIN-SRV-7702-UUID",
  "hostname": "winsrv7702",
  "gsma_code": "FR-LY0-DC2",
  "host_role_env": "prod",
  "platform": "windows",
  "os_family": "Windows",
  "os_distribution": "WindowsServer",

```

```
"os_version": "10.0.20348",
"architecture": "AMD64",
"snapshot_id": "snap-win-20260612T021500Z-7702",
"scan_timestamp": "2026-06-12T02:15:00Z",
"collection_method": "winrm-powershell",
"scan_confidence": 0.97,
"filesystems": [
  {
    "mount_point": "C:",
    "label": "System",
    "fstype": "ntfs",
    "size_total_bytes": 137438953472,
    "size_used_bytes": 64424509440,
    "size_available_bytes": 73014444032,
    "reserved_bytes": null,
    "inodes_total": null,
    "inodes_used": null,
    "access_model": "ntfs-acl",
    "mount_options": null,
    "acl_summary": {
      "owner": "BUILTIN\\Administrators",
      "inheritance_enabled": true,
      "ace_count": 6,
      "well_known_principals": ["SYSTEM", "BUILTIN\\Administrators", "BUILTIN\\Users"],
      "has_everyone_write": false
    },
    "backing_kind": "partition",
    "backing_ref": "disk0-part2",
    "group_ref": null,
    "content_aggregates": {}
  },
  {
    "mount_point": "D:",
    "label": "Data",
    "fstype": "refs",
    "size_total_bytes": 2199023255552,
    "size_used_bytes": 879609302220,
    "size_available_bytes": 1319413953332,
    "reserved_bytes": null,
    "inodes_total": null,
    "inodes_used": null,
    "access_model": "ntfs-acl",
    "mount_options": null,
    "acl_summary": {
      "owner": "BUILTIN\\Administrators",
```



```

        "inheritance_enabled": true,
        "ace_count": 4,
        "well_known_principals": ["SYSTEM", "BUILTIN\\Administrators", "APP\\DataOperators"],
        "has_everyone_write": false
    },
    "backing_kind": "virtual-disk",
    "backing_ref": "vdisk-data01",
    "group_ref": "Pool-Data",
    "content_aggregates": {}
}
],
"storage_topology": {
    "groups": [
        {
            "group_name": "Pool-Data",
            "group_kind": "storage-pool",
            "size_total_bytes": 4398046511104,
            "size_free_bytes": 2199023255552,
            "member_disk_count": 4,
            "state": "Healthy",
            "extent_size_bytes": null,
            "capabilities": { "max_lvs": null, "max_pps": null, "vg_type": null, "resiliency":
"Mirror" }
        }
    ],
    "volumes": [
        { "volume_name": "disk0-part2", "volume_kind": "partition", "group_ref": null, "size_bytes":
137438953472, "type": "gpt-part", "redundancy": "none" },
        { "volume_name": "vdisk-data01", "volume_kind": "virtual-disk", "group_ref": "Pool-Data",
"size_bytes": 2199023255552, "type": "Fixed", "redundancy": "mirror" }
    ],
    "disks": [
        { "disk_name": "Disk0", "size_bytes": 137975824384, "free_bytes": 0, "group_ref": null,
"media_type": "ssd" },
        { "disk_name": "PhysicalDisk1", "size_bytes": 1099511627776, "free_bytes": 549755813888,
"group_ref": "Pool-Data", "media_type": "hdd" },
        { "disk_name": "PhysicalDisk2", "size_bytes": 1099511627776, "free_bytes": 549755813888,
"group_ref": "Pool-Data", "media_type": "hdd" },
        { "disk_name": "PhysicalDisk3", "size_bytes": 1099511627776, "free_bytes": 549755813888,
"group_ref": "Pool-Data", "media_type": "hdd" },
        { "disk_name": "PhysicalDisk4", "size_bytes": 1099511627776, "free_bytes": 549755813888,
"group_ref": "Pool-Data", "media_type": "hdd" }
    ]
},
"paging": [

```

```
{ "name": "C:\\pagefile.sys", "kind": "windows-pagefile", "size_total_bytes": 17179869184,  
  "size_used_bytes": 2147483648 }  
]  
}
```